

Airport Security Simulation

How Many Security Personnel Required?

Simulate a simplified airport security system at a busy airport. Passengers arrive according to a Poisson distribution with $\lambda_1 = 5$ per minute (i.e., mean interarrival rate $\mu_1 = 0.2$ minutes) to the ID/boarding-pass check queue, where there are several servers who each have exponential service time with mean rate $\mu_2 = 0.75$ minutes. After that, the passengers are assigned to the shortest of the several personal-check queues, where they go through the personal scanner (time is uniformly distributed between 0.5 minutes and 1 minute).

Use Python with SimPy to build a simulation of the system, and then vary the number of ID/boarding-pass checkers and personal-check queues to determine how many are needed to keep average wait times below 15 minutes.

Methodology

I used Python's SimPy library to build a discrete-event simulation of a two-stage airport security system.

The model represents a 9-hour (540-minute) operational period to approximate a single "day shift." The first hour was treated as a warm-up period, during which queues start empty and system performance is excluded from steady-state results. The steady-state period begins after this initial hour.

The simulation includes the following main functions:

- `passenger_two_stage` – models a single passenger's journey through both stages of security, dynamically selecting the shortest Stage 2 queue and recording all timing data for later analysis.
- `passenger_arrivals_two_stage` – generates passenger arrivals following a Poisson process with exponential interarrival times, launching a new SimPy process for each passenger.
- `run_single_replication_two_stage` – executes one replication of the two-stage system for specified server counts, collecting detailed passenger-level wait and service statistics.
- `run_experiment_two_stage` – automates multiple replications across all Stage 1 and Stage 2 server configurations, combining results into a single dataset for analysis and visualization.

- `analyze_results_two_stage` – aggregates and summarizes outcomes, calculating average wait times, utilizations, and determining whether total average wait meets the 15-minute performance target.
- `validate_distribution_two_stage` – verifies that simulated service times follow the intended exponential (Stage 1) and uniform (Stage 2) distributions using both visual (Q–Q and histogram plots) and statistical (Kolmogorov–Smirnov test) validation.

Passenger arrivals were modeled with an exponential rate of $\lambda = 5$ passengers per minute. The simulation tested all combinations of 1–6 servers at each stage, running 50 independent replications per configuration.

Results included warm-up and steady-state averages, utilization levels, and total system wait times, which were then summarized in a comprehensive results table and plotted to visualize performance trends.

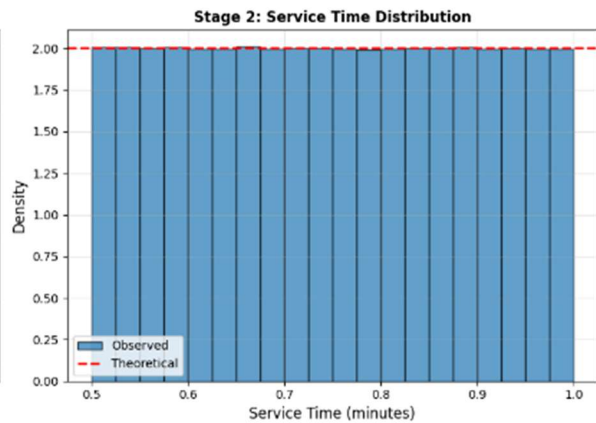
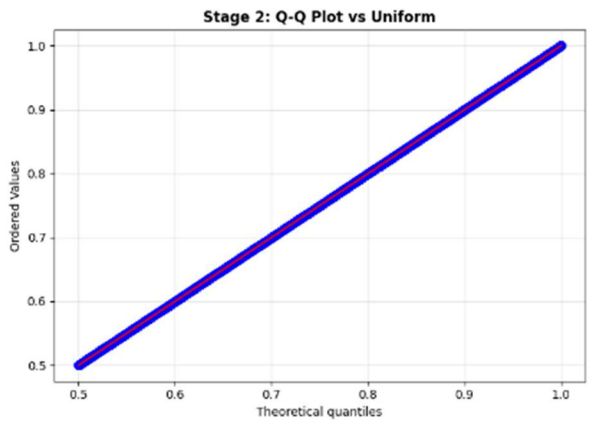
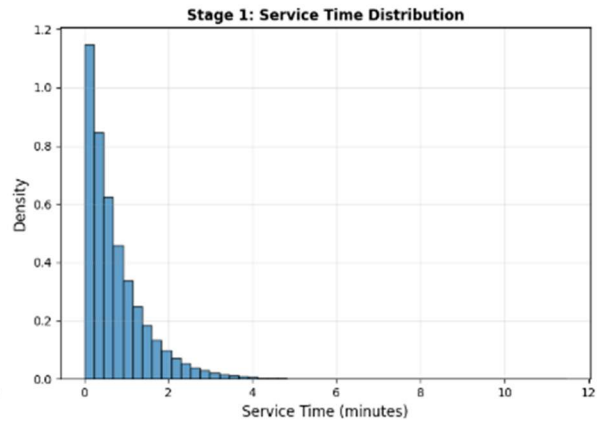
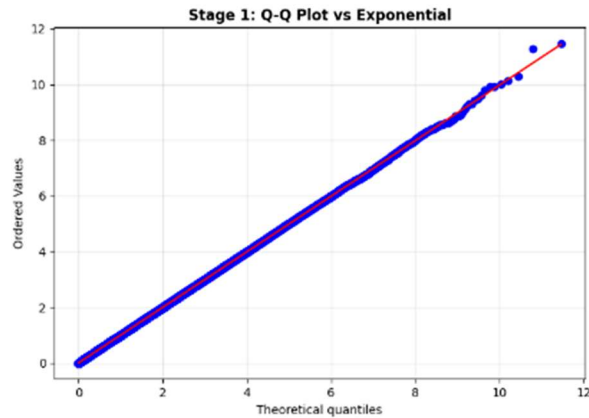
To validate model behavior, the Stage 1 interarrival and service times were compared to an exponential distribution ($\lambda = 5$, scale = 0.2) using Q–Q plots and histograms. Stage 2 service times were similarly compared to a Uniform(0.5, 1.0) distribution, with the theoretical probability density function (PDF) shown as a flat reference line. Kolmogorov–Smirnov test results were also computed and printed for both stages to statistically confirm alignment with the expected theoretical distributions.

Finally, just to compare, I tried multiple simulations with $\lambda_1 = 50$ for a busier airport scenario.

Results

Both service stages passed their statistical validation checks:

The simulated exponential service times are consistent with the theoretical exponential distribution and the uniform service times follow the intended flat probability density function.



Stage 1 (Exponential):

Observed mean: 0.7489 (expected: 0.7500)

KS statistic: 0.0008

P-value: 0.0664

✓ PASS: Data is consistent with Exponential distribution ($p > 0.05$)

Stage 2 (Uniform):

Observed mean: 0.7498 (expected: 0.7500)

KS statistic: 0.0007

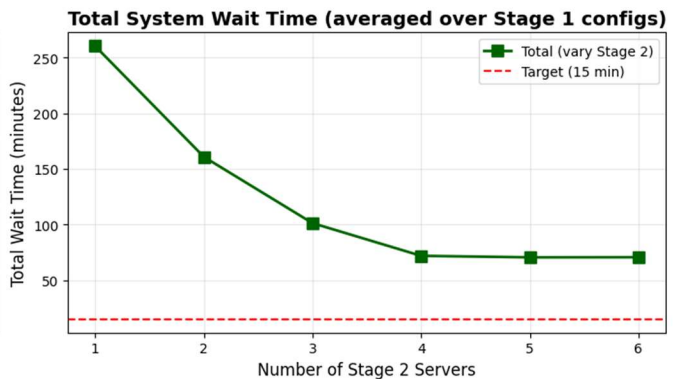
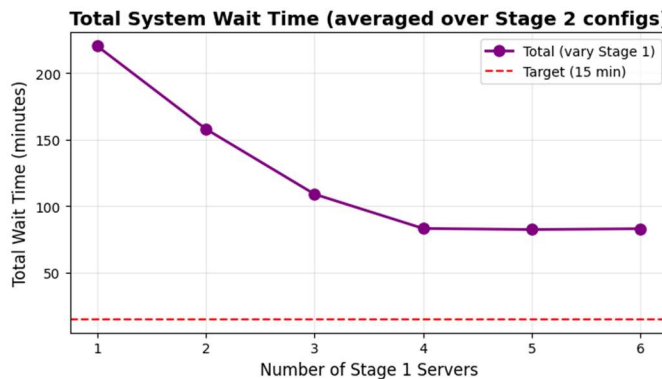
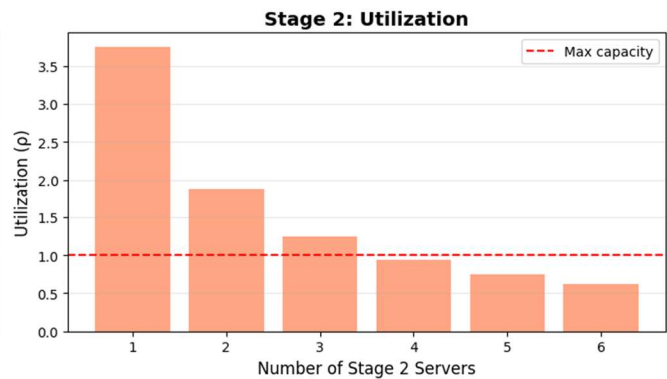
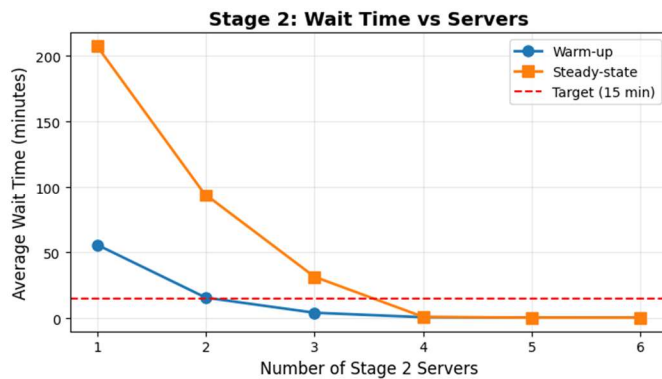
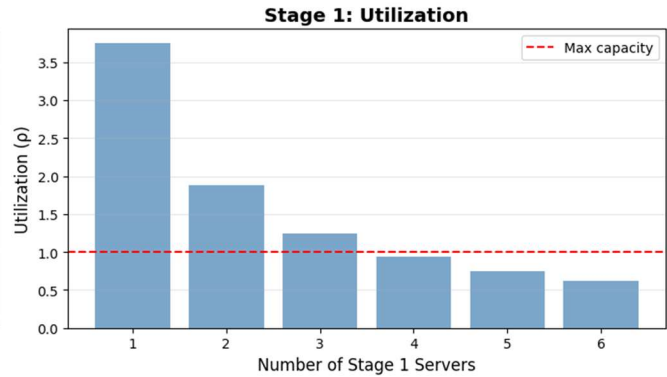
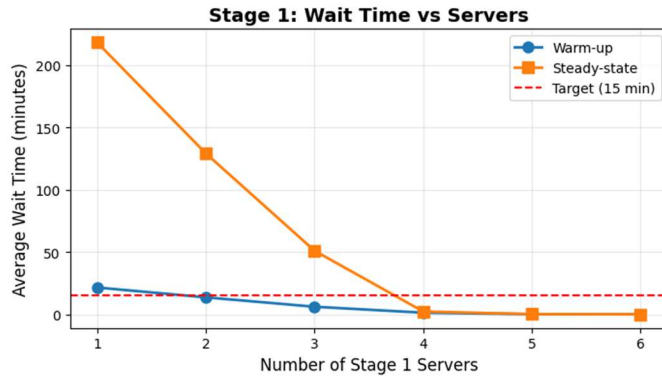
P-value: 0.1408

✓ PASS: Data is consistent with Uniform distribution ($p > 0.05$)

TWO-STAGE SYSTEM RESULTS

Stage 1 srvrs	Stage 2 srvrs	Util stage1	Util stage2	Warmup stage1 wait	Warmup stage2 wait	Warmup total wait	Steady stage1 wait	Steady stage2 wait	Steady total wait	meets target
1	1	3.75	3.75	22.008485	3.107954	25.116439	214.587758	11.595633	226.18339	✗
1	2	3.75	1.875	21.741045	0.305463	22.046508	220.069197	0.303577	220.372774	✗
1	3	3.75	1.25	21.70683	0.252582	21.959412	217.499193	0.251622	217.750815	✗
1	4	3.75	0.9375	22.154415	0.24162	22.396035	218.706652	0.247831	218.954483	✗
1	5	3.75	0.75	21.204312	0.248273	21.452585	219.265215	0.24947	219.514685	✗
1	6	3.75	0.625	20.803566	0.254545	21.05811	218.442296	0.24922	218.691516	✗
2	1	1.875	3.75	13.479101	29.716401	43.195502	76.78924	162.876967	239.666207	✗
2	2	1.875	1.875	13.646392	2.380095	16.026487	136.952262	7.948571	144.900833	✗
2	3	1.875	1.25	13.663269	0.440221	14.10349	139.391439	0.446837	139.838276	✗
2	4	1.875	0.9375	14.079075	0.354414	14.433489	141.759199	0.354762	142.113961	✗
2	5	1.875	0.75	13.510833	0.346156	13.856989	140.139227	0.346711	140.485938	✗
2	6	1.875	0.625	13.97639	0.343547	14.319937	139.837757	0.346454	140.184211	✗
3	1	1.25	3.75	6.093252	58.039143	64.132396	24.250894	236.534413	260.785307	✗
3	2	1.25	1.875	6.111493	14.676331	20.787824	43.661341	102.246678	145.908019	✗
3	3	1.25	1.25	5.95935	2.08512	8.04447	59.217556	6.590253	65.807809	✗
3	4	1.25	0.9375	6.127912	0.492667	6.620579	61.709347	0.531311	62.240658	✗
3	5	1.25	0.75	6.221535	0.413918	6.635453	57.669126	0.417527	58.086653	✗
3	6	1.25	0.625	6.612141	0.399983	7.012124	60.026416	0.403486	60.429902	✗
4	1	0.9375	3.75	1.391473	78.508913	79.900386	2.048089	274.512151	276.56024	✗
4	2	0.9375	1.875	1.256384	23.716507	24.972891	2.527741	148.554532	151.082274	✗
4	3	0.9375	1.25	1.230039	6.590265	7.820304	2.514763	58.501969	61.016733	✗
4	4	0.9375	0.9375	1.610098	0.96211	2.572208	2.359032	1.584231	3.943263	✓
4	5	0.9375	0.75	1.449401	0.478291	1.927692	2.624459	0.51276	3.137219	✓
4	6	0.9375	0.625	1.497049	0.435743	1.932792	2.235299	0.44032	2.675619	✓

Stage 1 srvrs	Stage 2 srvrs	Util stage1	Util stage2	Warmup stage1 wait	Warmup stage2 wait	Warmup total wait	Steady stage1 wait	Steady stage2 wait	Steady total wait	meets target
5	1	0.75	3.75	0.241996	81.257552	81.499548	0.244326	277.993268	278.237594	✗
5	2	0.75	1.875	0.236214	25.946029	26.182243	0.259338	150.259409	150.518747	✗
5	3	0.75	1.25	0.242062	7.913176	8.155237	0.26455	61.29204	61.55659	✗
5	4	0.75	0.9375	0.214165	1.084661	1.298827	0.263632	1.543537	1.807169	✓
5	5	0.75	0.75	0.203762	0.488317	0.692079	0.281252	0.510081	0.791333	✓
5	6	0.75	0.625	0.275692	0.442244	0.717936	0.278814	0.441808	0.720623	✓
6	1	0.625	3.75	0.091944	83.338125	83.430069	0.070446	281.016839	281.087285	✗
6	2	0.625	1.875	0.076015	26.399449	26.475465	0.07403	152.046267	152.120297	✗
6	3	0.625	1.25	0.079572	7.459896	7.539468	0.07517	61.201651	61.27682	✗
6	4	0.625	0.9375	0.062739	1.275617	1.338357	0.077451	1.685288	1.762738	✓
6	5	0.625	0.75	0.074583	0.4991	0.573683	0.070899	0.512981	0.583880	✓
6	6	0.625	0.625	0.081481	0.440325	0.521806	0.076162	0.440523	0.516685	✓



OPTIMAL CONFIGURATIONS

With the passenger arrival rate $\lambda_1 = 5$ (50 replications):

Minimum configuration meeting target (total wait < 15 min):

Stage 1 ID Check: 4 agents
 Stage 2 Screening: 4 lanes
 Stage 1 wait: 2.36 min
 Stage 2 wait: 1.58 min
 Total system wait: 3.94 min
 Total Servers: 8

With the passenger arrival rate $\lambda_1 = 50$ (5 replications):

Minimum configuration meeting target (total wait < 15 min):

Stage 1 ID Check: 36 agents



Stage 2 Screening: 36 lanes
Stage 1 wait: 12.33 min
Stage 2 wait: 2.35 min
Total system wait: 14.67 min
Total Servers: 72

Discussion of Results

The system was tested with 1–6 servers at both stages (36 configurations total).

The utilization figures reveal where congestion occurs:

- With few servers (e.g., 1–3 at Stage 1), utilization exceeds 100%, causing massive queues and delays (steady-state waits > 200 minutes).
- Increasing Stage 1 capacity drastically reduces total waits, confirming Stage 1 is the bottleneck.
- Once Stage 1 has at least 4 servers, total wait times drop sharply.
- Stage 2 utilization falls below 1.0 for most feasible configurations, indicating it is rarely the limiting factor once Stage 1 is balanced.

The goal was a total average steady-state wait < 15 minutes. Only configurations with Stage 1 ≥ 4 servers and Stage 2 ≥ 4 servers meet this target. Adding more than 4 servers to either stage gives diminishing returns — total waits are already well below the 15-minute threshold.

Therefore the recommended minimum configuration meeting the 15-minute target is 4 agents for the ID/ticket check and 4 screening/scanner lanes, for a total of 8 “servers.” The average total steady-state wait time with the 4x4 configuration was 3.94 minutes (2.36 minutes for Stage 1 and 1.58 minutes for Stage 2). With the passenger arrival rate of 5 per minute, this configuration balances service capacity efficiently, minimizing both wait time and total staffing while maintaining high service quality. Adding more servers yields only marginal improvements (e.g., total waits around 3 minutes) but increases staffing cost unnecessarily.

Exploratory analysis with $\lambda=50$ suggests approximately 36 agents per stage would be needed, though more replications would be required for precise estimates, as I only used 5 replications per server configuration due to computing resource constraints.